

Information Retrieval System

Milestone 2: Retrieval System & Experimental Framework

Corpus: 500 Documents | 8 Topics | 3 Retrieval Models

Abdullah Al Dahleh (20220385)
Tala Beirouti (20230450)

1. Introduction

This report documents the design, implementation, and experimental evaluation of an Information Retrieval (IR) system built for Milestone 2. The system operates on a corpus of 500 documents spanning eight technical topics and implements three retrieval models: Vector Space Model (VSM), BM25, and an Embedding-based model.

The GUI was built using Streamlit and supports spelling correction, abbreviation expansion, case-insensitive search, and exact phrase matching. Twenty evaluation queries were designed and tested across all three models.

2. Corpus Overview

Table 1 summarizes the topic distribution across the 500-document corpus.

Table 1: Corpus Topic Distribution

Topic	Document Count	Percentage (%)
Artificial Intelligence	93	18.16
Cloud Computing	81	15.82
Cybersecurity	67	13.09
Blockchain	60	11.72
Data Science	59	11.52
Quantum Computing	59	11.52
Internet of Things	47	9.18
Computer Networks	46	8.98
TOTAL	512	100.00

3. System Architecture

The system consists of five modular Python components:

- `corpus_loader.py` : Loads and normalizes the JSON/CSV corpus into a standard schema (id, title, content, topic, url).
- `query_handler.py` : Four-stage query pipeline: case normalization → spell correction → abbreviation expansion → phrase detection.
- `models/vsm.py` : TF-IDF weighted vectors with cosine similarity using scikit-learn. Bigrams enabled, sublinear TF scaling applied.
- `models/bm25.py` : Okapi BM25 using `rank_bm25` library with $k1=1.5$, $b=0.75$ and NLTK tokenization.

- `models/embedding.py` :Sentence Transformers (all-MiniLM-L6-v2) producing 384-dimensional dense embeddings, with first-run caching.
- `app.py` :Streamlit GUI with search box, model dropdown, Top-10 ranked results including title, URL, 200-character snippet, and relevance score.

4. Retrieval Models

4.1 Vector Space Model (VSM)

VSM represents documents and queries as TF-IDF weighted vectors and ranks by cosine similarity. Title text is duplicated to boost title-matching importance. Documents containing an exact phrase receive a 1.5x score multiplier.

- Strengths: Fast indexing and retrieval, fully interpretable scores, no training required.
- Weaknesses: Vocabulary-bound which cannot handle synonyms, paraphrases, or semantic similarity. No notion of term proximity.

4.2 BM25

BM25 improves over VSM by adding TF saturation (terms cannot score infinitely with repetition) and document-length normalization. Parameters $k_1=1.5$ and $b=0.75$ follow standard IR conventions.

- Strengths: Handles varying document lengths better than VSM; generally higher Precision@10 on keyword queries.
- Weaknesses: Still vocabulary-based; high-frequency terms in large topic groups can dominate results regardless of relevance.

4.3 Embedding-Based Retrieval

Sentence Transformers encode both documents and queries into dense semantic vectors. The all-MiniLM-L6-v2 model offers a strong balance between speed and accuracy. Embeddings are computed once and cached, making subsequent runs instantaneous.

- Strengths: Captures semantic meaning, handles paraphrases, synonyms, and natural language queries effectively.
- Weaknesses: General-purpose pre-training may not reflect domain-specific technical vocabulary precisely. Higher computational cost on first run.

5. Query Handling Features

- **Case Insensitivity:** All queries are lowercased before processing.
- **Spelling Correction:** pyspellchecker detects and fixes misspelled tokens. Quoted phrases and known abbreviations are protected from correction.
- **Abbreviation Expansion:** 21 domain-specific abbreviations (AI, IoT, NLP, QC, DeFi, etc.) are expanded to their full forms before retrieval.
- **Phrase Matching:** Tokens enclosed in double quotes trigger exact phrase detection. Matched documents receive a score boost.

6. Evaluation Query Design

Twenty queries were designed across four categories to stress-test different aspects of each retrieval model. Ground-truth relevance is defined by topic labels.

6.1 Keyword Queries (Q01–Q05)

ID	Query	Challenge	Relevant Topics
Q01	neural network deep learning	Short keywords match many docs across AI and Data Science.	AI, Data Science
Q02	blockchain consensus protocol	"Consensus" also appears in networking docs.	Blockchain, Networks
Q03	quantum entanglement qubit	Specialized vocabulary; rare co-occurrence.	Quantum Computing
Q04	cloud storage encryption	"Encryption" spans Cybersecurity and Cloud equally.	Cloud, Cybersecurity
Q05	IoT sensor network	Abbreviation must expand; "network" overlaps Computer Networks.	IoT, Networks

6.2 Natural Language Queries (Q06–Q10)

ID	Query	Challenge	Relevant Topics
Q06	How does ML improve cybersecurity threat detection?	Cross-topic; long question reduces keyword overlap.	AI, Cybersecurity
Q07	What are challenges of blockchain in supply chains?	"Supply chain" may not appear verbatim in corpus.	Blockchain
Q08	How do quantum computers outperform classical computers?	"Outperform" is not a typical technical keyword.	Quantum Computing
Q09	Difference between public and private cloud deployment?	Docs may not directly compare the two models.	Cloud Computing
Q10	How can data science detect network intrusions?	Cross-topic bridge between Data Science and Cybersecurity.	Data Science, Cybersecurity

6.3 Ambiguous Queries (Q11–Q15)

ID	Query	Challenge	Relevant Topics
Q11	network security	"Network" could mean blockchain, computer network, or neural network.	Cybersecurity, Networks, Blockchain
Q12	distributed systems	Equally relevant to Blockchain, Cloud, and Computer Networks.	Blockchain, Cloud, Networks
Q13	data mining	"Mining" = crypto mining OR data mining technique.	Data Science, Blockchain
Q14	smart contracts automation	"Smart" may pull AI docs; "automation" overlaps IoT.	Blockchain, IoT
Q15	edge computing latency	"Edge" key in IoT and Cloud; "latency" is generic.	IoT, Cloud, Networks

6.4 Noisy / Typo Queries (Q16–Q20)

ID	Query	Challenge	Relevant Topics
Q16	mchine lerning algorithms	Three typos — machine, learning, algorithms.	AI, Data Science
Q17	blokchain cryptocurency trnsaction	Three misspellings in domain vocabulary.	Blockchain
Q18	cybr securtiy atack prevntion	Heavy noise; all four words contain errors.	Cybersecurity
Q19	quantm computng qubit procesing	Rare technical vocabulary made harder by misspellings.	Quantum Computing
Q20	clod computng servr virtualiztion	"clod" may confuse spell checker.	Cloud Computing

7. Experimental Results

All three models were evaluated on the 20 queries using Precision@10, Recall, and Mean Average Precision (MAP). Ground-truth relevance was defined by topic labels: a document is relevant to a query if its topic appears in the query's predefined relevant-topic set. Table 5 presents the results.

7.1 Overall Model Comparison

Table 5: Overall Model Performance (averaged over 20 queries)

Model	Precision@10	Recall	MAP
VSM (TF-IDF)	0.780	0.0812	0.0694
BM25	0.830	0.0895	0.0826
Embedding-based	0.865	0.0935	0.0892

7.2 Analysis of Results

The Embedding-based model achieves the highest scores across all three metrics ($P@10=0.865$, $Recall=0.0935$, $MAP=0.0892$). BM25 ranks second and significantly outperforms VSM, while VSM remains the weakest performer across all metrics.

The low absolute Recall values (all below 0.10) are expected and are a direct consequence of how ground-truth relevance is defined in this evaluation. A query such as Q01 ("neural network deep learning") has 152 relevant documents in the corpus (all AI + Data Science documents by topic label), but the system only retrieves 10. This structural ceiling limits Recall regardless of model quality.

MAP values track closely with Precision@10, confirming that the models which rank correctly also maintain high precision at earlier ranks. The Embedding model shows the largest advantage on natural language queries (Q06–Q10) and noisy queries (Q16–Q20), where lexical matching is insufficient and semantic understanding is required.

7.3 Per-Query-Type Observations

- Keyword queries (Q01–Q05): BM25 and VSM perform competitively since exact term overlap is sufficient. The Embedding model adds marginal improvement.
- Natural language queries (Q06–Q10): The Embedding model shows the clearest advantage. VSM and BM25 struggle when query phrasing does not match document vocabulary.
- Ambiguous queries (Q11–Q15): All models face cross-topic noise. BM25 handles frequency-dominant topics better than VSM. Embedding returns more topically diverse results.
- Noisy queries (Q16–Q20): After spell correction, all models recover well. The Embedding model is most resilient when corrections are imperfect.

8. Failure Case Analysis

A failure case is defined as a scenario where a model returns results that, despite technically matching surface-level keywords, are not relevant to the user's information need or where the model returns fewer relevant results than expected given the query. The following three cases were identified through targeted adversarial queries designed to expose each model's known weaknesses.

8.1 VSM Failure Case — Vocabulary Mismatch (Synonym Query)

Failure Query: *"protecting sensitive user data from unauthorized breaches"*

Expected behavior: The query clearly describes a Cybersecurity topic, protecting data from breaches. The system should return documents about intrusion detection, encryption, data security, and access control.

Actual behavior: VSM returns low-confidence or incorrect results, with Computer Networks and Cloud Computing documents appearing in the top 10. The relevant Cybersecurity documents are ranked much lower or absent entirely.

Why it fails: VSM relies entirely on exact term overlap between query and document. Cybersecurity documents in the corpus use vocabulary such as: 'threat', 'malware', 'intrusion', 'cyberattack',

'firewall', and 'vulnerability'. None of these words appear in the query. The query uses 'protecting', 'sensitive', 'unauthorized', and 'breaches', valid synonyms that carry the same meaning but are statistically absent from the TF-IDF vocabulary of the relevant documents. Cosine similarity is near zero for the most relevant documents.

Root cause: TF-IDF has no mechanism for synonym resolution or semantic equivalence. The vocabulary mismatch between query and document is the fundamental limitation of bag-of-words models like VSM.

Suggested fix: Query expansion using WordNet or a domain thesaurus to add synonyms (e.g., 'protecting' > add 'securing', 'safeguarding'; 'breaches' > add 'attacks', 'intrusions') before retrieval.

8.2 BM25 Failure Case — Term Frequency Bias Across Topics

Failure Query: *"network architecture design"*

Expected behavior: The query targets Computer Networks documents discussing how networks are structured and designed (topology, routing protocols, layered architecture, etc.).

Actual behavior: BM25 returns a mix of Computer Networks, IoT, Blockchain, and Artificial Intelligence documents. The top results include IoT documents (which discuss 'sensor network architecture') and AI documents (which discuss 'neural network architecture'). Relevant Computer Networks documents are diluted by these cross-topic matches.

Why it fails: The word 'network' is extremely high-frequency across multiple topics in this corpus. IoT documents use 'sensor network', Blockchain documents use 'peer-to-peer network', AI documents use 'neural network architecture', and Cloud documents use 'virtual network'. BM25's length normalization reduces the effect of document length, but it cannot distinguish the semantic context of 'network'. Any document with all three words (network, architecture, design) in any combination scores high, regardless of whether they appear together as a meaningful concept.

Root cause: BM25 scores terms independently with no positional or contextual awareness. High cross-topic frequency of the term 'network' (the single most common technical term in this corpus) creates systematic false positives across all topics.

Suggested fix: Use positional BM25 (BM25F with field weights) or apply a topic-based filter. Alternatively, phrase-match the entire query by wrapping it in quotes: "network architecture design".

8.3 Embedding Model Failure Case — Semantic Over-Generalization

Failure Query: *“decentralized intelligent data-driven secure interconnected computational systems”*

Expected behavior:

The query contains strong indicators of Artificial Intelligence (“intelligent”, “data-driven”) and should primarily retrieve AI-related documents. Secondary results may include: ,Cybersecurity (“secure”) ,IoT / Computer Networks (“interconnected”) ,Cloud Computing (“computational systems”) .The expected outcome is a dominant AI cluster, with limited spillover into closely related domains

Actual behavior: The top 10 retrieved documents are heavily skewed toward a subset of topics:
Cloud Computing (~5 documents)
Blockchain (~4 documents)
Quantum Computing (1 document)

Why it fails: Although the query is multi-domain, the embedding model does not treat all terms equally. Instead, it exhibits semantic bias toward dominant or strongly clustered concepts: “decentralized” strongly pulls toward Blockchain, “computational systems” and “scalable infrastructure” associations pull toward Cloud Computing. These domains tend to have denser or more cohesive embedding clusters in many corpora

Meanwhile:

AI-related terms (“intelligent”, “data-driven”) are too generic and appear across multiple domains
This weakens their ability to anchor the query in the AI cluster

As a result, the query embedding is pulled away from AI and toward Cloud + Blockchain regions, where semantic signals are stronger or more specific.

Root cause: This failure arises from uneven semantic dominance in embedding space, where: Some terms have strong, domain-specific signals (e.g., “decentralized” → Blockchain) Others are overly generic and diffuse (e.g., “intelligent”, “data-driven”). The model implicitly weights more distinctive concepts higher

This leads to Cluster bias instead of uniform averaging, suppression of weaker semantic signals (AI in this case), retrieval dominated by a few “gravitational” topics

Suggested fix: Use more specific query language or implement a query clarification step that asks the user to specify the domain context before retrieval.

9. Discussion

The three models represent a tradeoff between specificity, semantic coverage, and computational cost:

Criterion	VSM	BM25	Embedding
P@10 (this corpus)	0.780	0.830	0.865
Recall (this corpus)	0.0812	0.0895	0.0935
MAP (this corpus)	0.0694	0.0826	0.0892
Indexing speed	Fast	Fast	Slow (first run only)
Query speed	Fast	Fast	Medium
Handles synonyms	No	No	Yes
Handles typos	No	No	Partial
Handles natural language	Partial	Partial	Yes
Interpretability	High	High	Low
Risk of over-generalization	Low	Low	High

The Embedding model consistently outperforms the lexical models across all metrics. However, its advantage decreases for highly specific technical queries and disappears entirely for semantically broad queries. A practical hybrid system would route short keyword queries to BM25 (faster, more precise) and natural-language or noisy queries to the Embedding model.

10. Conclusion

This milestone implemented a complete Information Retrieval system with three models, a Streamlit GUI, four query handling capabilities, and a 20-query experimental framework. Experimental results show Embedding > BM25 > VSM across Precision@10, Recall, and MAP on this corpus.

Each model exhibits a distinct, reproducible failure mode: VSM fails on vocabulary-mismatched synonym queries, BM25 fails when a high-frequency cross-topic term dominates scoring, and the Embedding model fails on semantically broad abstract queries. These failure modes are not bugs — they are the fundamental tradeoffs of each retrieval paradigm, and understanding them is essential for designing robust hybrid IR systems.

Appendix: Complete Query List

ID	Type	Query	Relevant Topics
Q01	Keyword	neural network deep learning	AI, Data Science
Q02	Keyword	blockchain consensus protocol	Blockchain, Networks
Q03	Keyword	quantum entanglement qubit	Quantum Computing
Q04	Keyword	cloud storage encryption	Cloud, Cybersecurity
Q05	Keyword	IoT sensor network	IoT, Networks
Q06	Natural Lang	How does machine learning improve cybersecurity threat detection?	AI, Cybersecurity
Q07	Natural Lang	What are the main challenges of implementing blockchain in supply chains?	Blockchain
Q08	Natural Lang	How do quantum computers outperform classical computers?	Quantum Computing
Q09	Natural Lang	What is the difference between public and private cloud deployment?	Cloud Computing
Q10	Natural Lang	How can data science be used to detect network intrusions?	Data Science, Cybersecurity
Q11	Ambiguous	network security	Cybersecurity, Networks, Blockchain
Q12	Ambiguous	distributed systems	Blockchain, Cloud, Networks
Q13	Ambiguous	data mining	Data Science, Blockchain
Q14	Ambiguous	smart contracts automation	Blockchain, IoT
Q15	Ambiguous	edge computing latency	IoT, Cloud, Networks
Q16	Noisy	machine learning algorithms	AI, Data Science
Q17	Noisy	blockchain cryptocurrency transaction	Blockchain
Q18	Noisy	cyber security attack prevention	Cybersecurity
Q19	Noisy	quantum computing qubit processing	Quantum Computing
Q20	Noisy	cloud computing server virtualization	Cloud Computing